Elaborar un informe, que indique qué maq. Están conectadas a la red, con el nombr. de user (x ej, 1 dict: key: machine, value: current_user), partiendo de que la empresa ya dispone de la clase "Events", que incluye: machine, user, date, type (= "login" o "logout").
Nota: tener en cuenta, que debe reflejarse el último usuario conectado y por tanto, hay que ordenar cromológica/)

A)

```
def get_event_date(event):
    return event.date

def current_users(events):
    machines = {}
    events.sort(key = get_event_date)
    for event in events:
        if event.machine not in machines:
            machines[event.machine] = set()
# Si no pongo instrucciones anteriores, al borrarse las de logout, ya no se recuperan. Aunque borre solo la parte "value" del dict (mirar comentario de logout)
# configuro dict como conj. vacío, para luego ir rellenándolo y para eliminar users repes, para cada maq.

# Si meto estos 2 if siguientes, dentro del anterior:
  #1)no se queda ninguna maq sin user. Si no hay login, deja logout
  #2)Si hay 2 login, deja el 1º q se conectó
        if event.type == "login":
            machines[event.machine].add(event.user)   #cada maq, lista de usuarios => agregar los nuevos conectados y borrar los antiguos


        elif event.type == "logout":
            machines[event.machine].remove(event.user)   #
    return machines

def generate_report():
    machines = current_users(events)
    for machine, users in machines.items():
        if len(users) > 0:
            users_list = ", ".join(users)
            print("{}: {}".format(machine, users_list))

class Event:
    def __init__(self,event_date, event_type ,machine_name,  user):
        self.date = event_date
        self.type = event_type
        self.machine = machine_name
        self.user = user


events = [
```

AutomySeg

```
    Event('2020-01-21 12:45:56', 'login', 'myworkstation.local','jordan'),    # fecha es 1 cad. Tb typo
=> no usar palabra reservada:type (creo; al - tiene otro color)
    Event('2020-01-22 15:53:42', 'logout', 'webserver.local', 'jordan'),
    Event('2020-01-21 18:53:21', 'login', 'webserver.local', 'lane'),
    Event('2020-01-22 10:25:34', 'logout', 'myworkstation.local', 'jordan'),
    Event('2020-01-21 08:20:01', 'login', 'webserver.local', 'jordan'),
    Event('2020-01-23 11:24:35', 'login', 'mailserver.local', 'Chris'),
]

# Una vez definido todo, programa princ. (como void main (void) : llamo a las funciones que
necesito para obtener el resultado. O en SPS, en el princ., llamo a las funciones (call...))
machines_users = current_users(events)
print(machines_users)  # = print(current_users(events))
print()
generate_report()
  ---------------------------------------------------------------------------------------------------------------
 {'webserver.local': {'lane'}, 'myworkstation.local': set(), 'mailserver.local': {'chris'}}

webserver.local: lane
mailserver.local: chris
```

B) = que anterior, pero en events cambia el último event; el de Chris, que en lugar de "login", es "logout"

Q. Ocurre, ahora? → Sale error: keyerror: ...Chris → por q? →

Descubre Chris no no fue 1º "login" (al no ser añadido en el conj., no se puede borrar) → Sol:

cambiar función remove x **dirscard** (esta, si conj. vacío, no da error; Simplemente, no hace nada)

AutomySeg